

UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

Facultad de Ingeniería

Ingeniería en Desarrollo de Software y Ciencia de Datos

Programación II

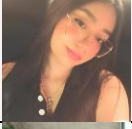
Avance 2

Sistema para Administración de Asociación Comunal

Grupo 10

Facilitador: Ing. Daniel Enrique Guevara Gómez

San Salvador, El Salvador

Foto	Nombre completo	CIF	¿Participó?
	Isabel Uvaldina Ramos Saravia	2026010478	NO
	Marcos Armando Vásquez González	2026011588	Sí
	Anderson Steven Flores Pérez	2026011705	Sí
	Camila Nicole Salvador Samayoa	2026011056	NO
	Luis Gerardo Ramos Martínez	2026010771	Sí
	Perla Esmeralda López Barrientos	2026011595	NO
	Diego Emanuel Escobar Hernández	2026011594	NO
	Joshua Alejandro Alas Rodriguez	2025010271	NO
	Roberto De Jesus Erazo Ramirez	2026010073	Sí
	Luis Alonso Alvarenga Meza	2026010120	NO

Objetivo general

Desarrollar un sistema informático para la administración de una asociación comunal que permita a la directiva y a los vecinos gestionar usuarios, proyectos, votaciones, secretaría y tesorería, y que notifique por correo electrónico los hechos que competen a cada uno, con el fin de centralizar la organización interna y el manejo de la información de la comunidad.

Este objetivo actualiza el planteado en el avance 1, que se centraba en el padrón de socios y el control de cuotas. Desde entonces, el alcance del sistema creció: ahora el vecino participa en los proyectos y la directiva gestiona la secretaría, la tesorería y la comunicación con la comunidad.

Objetivos específicos de este avance

- Actualizar el modelo del avance 1: el socio pasa a representarse como vecino, la directiva como miembro con cargo, y el rol deja de ser un dato independiente para derivarse del tipo de usuario.
- Implementar el backend en Java con Spring Boot, aplicando programación orientada a objetos, una arquitectura por capas y persistencia en archivos .dat.
- Notificar por correo electrónico, en segundo plano y mediante hilos, los hechos relevantes para los vecinos o la directiva, sin retrasar la operación que ya fue guardada.
- Construir el frontend en Next.js, con paneles diferenciados para el vecino y para la directiva, y con la sesión almacenada en una cookie HttpOnly.

El objetivo específico del avance 1 (registro y gestión de socios, ahora vecinos) se mantiene como base del sistema. El tercer objetivo específico del proyecto se abordará en el siguiente avance, cuya fecha de entrega aún no ha sido definida por la cátedra.

Distribución del equipo

Los roles Scrum asignados en el avance 1 se mantienen sin cambios. Isabel Uvaldina Ramos Saravia continúa como coordinadora del grupo en el rol de Scrum Master, aunque no participó en esta entrega. El trabajo fue realizado por los cuatro integrantes activos y, dado lo reducido del equipo, se distribuyó por módulos en lugar de seguir los roles técnicos originales.

N°	Nombre completo	Rol Scrum	Rol técnico	¿Participó?
1	Marcos Vásquez	Product Owner (Business Owner)	Analista	Sí
2	Isabel Ramos	Scrum Master	Desarrolladora	NO
3	Roberto Erazo	Business Analyst	Analista de negocio	Sí
4	Luis Gerardo Ramos	Architect	Arquitecto	Sí
5	Anderson Flores	Developer	Interfaz (frontend)	Sí
6	Luis Alonso Alvarenga	Developer	Backend	NO
7	Camila Salvador	Developer	Base de datos	NO
8	Esmeralda López	Developer	Fullstack	NO
9	Joshua Alas	QA	Analista de pruebas	NO
10	Diego Escobar	QA	Analista de pruebas	NO

Reparto real de este avance

Integrante	Qué hizo en este avance
Luis Gerardo Ramos Martínez	Arquitectura del frontend y del backend, flujo de autenticación, persistencia en archivos .dat y módulos de tesorería, secretaría y votaciones.
Marcos Armando Vásquez González	Envío de correos electrónicos mediante hilos desde Spring Boot, a través de un servidor SMTP.
Anderson Steven Flores Pérez	CRUD de proyectos en el frontend y en el backend.
Roberto De Jesus Erazo Ramirez	CRUD de usuarios en el frontend y en el backend.

Roles y funciones del sistema

En el avance 1 se definieron los roles de administrador, secretario, tesorero y socio. En la implementación, el vecino se modeló como una clase propia y la directiva como un miembro con cargo. El cargo determina en qué módulos puede escribir cada miembro: el presidente tiene acceso a secretaría y tesorería, el secretario solo a secretaría y el tesorero solo a tesorería.

Rol	Descripción	Funciones en el sistema
Vecino	Miembro de la comunidad. Consulta información y participa, pero no administra.	Iniciar sesión, ver los proyectos publicados, votar una vez por proyecto, comentar, enviar solicitudes, leer comunicados y actas publicadas, y consultar su estado en tesorería.
Presidente	Miembro de la directiva que coordina la operación de la asociación.	Gestionar proyectos, escribir en secretaría (actas, solicitudes y comunicados) y en tesorería (cuotas, pagos y gastos), y consultar votos y comentarios.
Secretario	Miembro de la directiva encargado del registro formal de la asociación.	Publicar y editar actas, atender solicitudes vecinales, publicar comunicados y consultar proyectos, votos y comentarios. No registra movimientos de tesorería.
Tesorero	Miembro de la directiva encargado de administrar los fondos de la asociación.	Registrar cuotas, pagos, gastos y movimientos, y consultar el balance. No publica actas ni comunicados.

Requerimientos del sistema

Las siguientes historias de usuario reemplazan a las del avance 1, que abarcaban el padrón de socios, las cuotas y los reportes de morosidad. Las actuales describen la funcionalidad implementada en esta entrega.

ID	Prioridad	Historia	Criterios de aceptación
HU-001	Alta	Como directiva, quiero registrar y consultar usuarios (vecinos y miembros de la junta) para controlar quién accede al sistema.	No se permiten correos duplicados. La cuenta queda registrada y el listado muestra nombre, correo y rol. Un vecino no puede administrar usuarios.
HU-002	Alta	Como directiva, quiero crear, consultar, editar y eliminar proyectos para publicar las iniciativas que la comunidad debe decidir.	El proyecto se registra con nombre, descripción, estado y fecha, aparece en el listado y puede modificarse o eliminarse.
HU-003	Alta	Como vecino, quiero votar una sola vez en cada proyecto abierto para participar en la decisión.	Solo un vecino con sesión iniciada puede votar. El voto queda asociado al proyecto y al correo del vecino, y un segundo voto del mismo vecino en el mismo proyecto no se registra.
HU-004	Alta	Como vecino, quiero enviar una solicitud a la directiva y consultar su respuesta, sin depender de un grupo de chat.	La solicitud registra título, descripción y autor. El presidente o el secretario pueden responderla, y el vecino puede ver su estado y la respuesta.
HU-005	Media	Como presidente o secretario, quiero publicar comunicados y actas para dejar constancia de lo acordado.	El comunicado o el acta se almacena con fecha y autor. El vecino solo ve lo publicado, y quien no tiene uno de esos cargos no puede escribir en secretaría.
HU-006	Alta	Como presidente o tesorero, quiero registrar cuotas, pagos y gastos para conocer el estado financiero de la asociación.	Cada cuota queda asociada a un vecino, un periodo y un monto. Registrar un pago actualiza su estado y cada movimiento se refleja en el balance. Ningún otro cargo puede escribir en tesorería.
HU-007	Media	Como vecino, quiero comentar un proyecto para dejar observaciones que la directiva pueda leer.	El comentario registra autor, texto, fecha y proyecto, y la directiva puede verlo en su panel. No se guardan comentarios vacíos.

HU-008	Media	Como vecino o miembro de la directiva, quiero recibir un aviso por correo cuando ocurra un hecho que me compete, sin tener que esperar en la pantalla.	El correo se encola después de guardar el dato y se envía en un hilo aparte. Si el servidor SMTP falla, el registro ya guardado se conserva y el intento queda anotado en el historial de envíos.
--------	-------	--	---

Alcances y limitaciones

Alcances de este avance

- Inicio de sesión con JWT: la sesión se comparte entre Next.js y Spring Boot, y el token se guarda en una cookie HttpOnly.
- Dos paneles: /comunidad para el vecino y /administracion para la directiva. La ruta de acceso depende del rol.
- CRUD de usuarios y CRUD de proyectos.
- Votación de proyectos y comentarios de la comunidad.
- Secretaría: solicitudes, comunicados y actas.
- Tesorería: cuotas, registro de pagos, gastos y balance. Los pagos se registran manualmente; no hay pasarela de pago.
- Envío de correos en segundo plano al recibir o responder una solicitud, al publicar un comunicado, ante una cuota pendiente y al registrar un pago.
- Persistencia en archivos .dat, con un archivo por cada entidad principal.

Respecto al avance 1, el alcance se amplió: las votaciones, los proyectos y las notificaciones, que aquel documento dejaba fuera, ya forman parte del sistema.

Limitaciones: qué no hace el sistema

- No procesa cobros en línea, ya que no cuenta con pasarela de pago; el tesorero registra los pagos manualmente.
- No permite administrar varias asociaciones en una misma instalación.
- No cuenta con aplicación móvil.
- No utiliza base de datos en este avance, ya que la consigna exige persistencia en archivos .dat. El uso de PostgreSQL, SQL Server u Oracle queda fuera de esta entrega.
- Un fallo en el envío de correo no revierte los datos guardados: el vecino puede consultar la información en el sistema aunque el servidor SMTP no responda.
- La cátedra aún no ha definido el objetivo específico número 3 del proyecto ni la fecha del avance 3.

Planificación

El desarrollo de este avance se llevó a cabo entre el 11 y el 27 de septiembre de 2026, y la entrega está programada para el 27 de septiembre de 2026 a las 13:00. Solo se incluyen las actividades de los integrantes que participaron.

Actividad	Responsable	Fecha
Arquitectura del frontend y del backend, autenticación, persistencia en archivos .dat, tesorería, secretaría y votaciones.	Luis Gerardo Ramos Martínez	11 al 27 de septiembre de 2026
Envío de correos mediante hilos y servidor SMTP.	Marcos Armando Vásquez González	11 al 25 de septiembre de 2026
CRUD de proyectos en el frontend y el backend.	Anderson Steven Flores Pérez	24 de septiembre de 2026
CRUD de usuarios en el frontend y el backend.	Roberto De Jesus Erazo Ramirez	24 de septiembre de 2026
Elaboración del documento del avance 2 y cierre de la entrega.	Luis Gerardo Ramos Martínez, con revisión de los cuatro integrantes activos	26 al 27 de septiembre de 2026

Cronograma

En el avance 1 se estableció que el proyecto se desarrollaría entre el 30 de julio y el 6 de noviembre de 2026, en tres avances, y ese horizonte se mantiene como referencia. La fecha del avance 3 no se incluye porque la cátedra aún no la ha publicado.

Entrega	Julio	Agosto	Septiembre	Octubre	Noviembre
Avance 1 30 jul al 6 ago	■	■			
Avance 2 11 al 27 sep			■		
Avance 3 Fecha no especificada			Pendiente		

Entradas y salidas

HU-001. Usuarios

Entradas: nombre, correo y contraseña, además del número de casa (si es vecino) o del cargo (si es miembro de la directiva).

Salidas: confirmación del registro, listado de usuarios y aviso en caso de que el correo ya exista.

HU-002. Proyectos

Entradas: nombre, descripción, estado y fecha de inicio.

Salidas: proyecto guardado, listado actualizado y ficha para consultarlo, editarlo o eliminarlo.

HU-003. Votación

Entradas: identificador del proyecto, correo del vecino con sesión iniciada y sentido del voto (a favor o en contra).

Salidas: voto registrado, totales a favor y en contra visibles para la directiva, y aviso si el vecino ya había votado.

HU-004. Solicitudes

Entradas: título y descripción; al responder, el texto de la respuesta y el nuevo estado.

Salidas: la solicitud aparece en el listado del vecino y en la bandeja de secretaría, y la respuesta queda visible para su autor.

HU-005. Comunicados y actas

Entradas: título, contenido, fecha y, en el acta, los acuerdos.

Salidas: comunicado o acta publicada, visible para los vecinos cuando corresponde.

HU-006. Tesorería

Entradas: vecino, periodo, monto y estado de la cuota; concepto, tipo, monto y fecha del movimiento.

Salidas: cuota actualizada, movimiento registrado en el historial y balance de ingresos y egresos.

HU-007. Comentarios

Entradas: proyecto, texto y autor (obtenido de la sesión).

Salidas: comentario visible en el proyecto y en el panel lateral de comentarios recientes.

HU-008. Correo

Entradas: destinatario, asunto y cuerpo, generados por la plantilla correspondiente al hecho (solicitud, comunicado, cuota o pago).

Salidas: registro del envío con su estado, número de intentos y error, si lo hubo. La pantalla del usuario no espera la respuesta del servidor SMTP.

Entidades del sistema

En el avance 1 se definieron tres entidades (Socio, Cuota y Usuario), y el rol era un atributo de la cuenta. En este avance, Usuario pasa a ser una clase abstracta de la que heredan Vecino y MiembroDirectiva, y se incorporan entidades para proyectos, participación, secretaría, tesorería y correo.

Entidad	Descripción	Atributos
Usuario	Cuenta de acceso. Clase abstracta; no se instancia directamente.	id, nombre, correo, password. Método abstracto getRol().
Vecino	Miembro de la comunidad. Hereda de Usuario.	numeroCasa. getRol() devuelve VECINO.
MiembroDirectiva	Integrante de la junta. Hereda de Usuario.	cargo (presidente, secretario o tesorero). getRol() se deriva del cargo.
Proyecto	Iniciativa o decisión que la directiva publica.	id, nombre, descripcion, estado, fechaInicio.
Voto	Participación de un vecino en un proyecto.	id, proyectold, correoUsuario, valor.
Comentario	Observación de un vecino sobre un proyecto.	id, proyectold, correoAutor, nombreAutor, texto, fecha.
Solicitud	Pedido de un vecino a la secretaría.	id, titulo, descripcion, correoAutor, nombreAutor, estado, respuesta, proyectold, fecha.
Comunicado	Aviso oficial de la directiva.	id, titulo, contenido, fecha, correoAutor, nombreAutor.
Acta	Constancia de una reunión y sus acuerdos.	id, titulo, contenido, fecha, estado, correoAutor, nombreAutor, acuerdos.
AcuerdoActa	Punto acordado dentro de un acta.	texto, proyectold.
Cuota	Aporte de un vecino correspondiente a un periodo.	id, correoVecino, nombreVecino, numeroCasa, periodo, monto, estado, fechaPago.
Movimiento	Ingreso o egreso de tesorería.	id, tipo, concepto, monto, fecha, proyectold, correoRegistro, nombreRegistro, cuotald.
RegistroEmail	Bitácora de cada correo encolado.	id, destinatario, asunto, fechaCreacion, estado, intentos, fechaEnvio, ultimoError, hiloProcesador.
PlantillaEmail	Clase abstracta base de los correos. No se persiste; cada hecho tiene su propia subclase.	destinatario. Métodos abstractos construirAsunto() y construirCuerpo().

Proyecto

Elección de tecnología

El backend está desarrollado en Java 21 con Spring Boot, y el frontend en Next.js (sobre React) con App Router. Ambos se comunican mediante una API REST en formato JSON. Esta combinación corresponde a la opción 6 de la guía del curso (Spring Boot + React). Se eligió Next.js porque permite preparar las páginas del panel en el servidor y porque el inicio de sesión puede comunicarse con Spring Boot de servidor a servidor.

Aunque la guía recomienda PostgreSQL, SQL Server u Oracle, en este avance no se conecta una base de datos, ya que la consigna exige persistencia en archivos .dat.

Evolución del diseño de clases del avance 1

En el avance 1, el paquete com.sv.grupo10.asociacioncomunal contenía las capas model, dao y service, junto con un Main de consola. El backend actual conserva la separación por capas, ahora dentro de Spring Boot, y el frontend se ubica en la carpeta web-asociacion-comunal del mismo repositorio. Socio dejó de ser la entidad central y fue reemplazado por Vecino. Además, Usuario ya no almacena el rol como un texto editable: cada subclase lo determina.

Encapsulamiento

Los atributos de las entidades son privados, por lo que cualquier lectura o modificación se realiza a través de métodos. En Usuario, los campos id, nombre, correo y password no pueden alterarse desde fuera de la clase. En RegistroEmail, el destinatario, el asunto y la fecha de creación se declaran como final: se asignan al crear el registro

y no vuelven a modificarse. En cambio, el estado, los intentos y el último error sí cambian, porque reflejan el resultado del envío.

Herencia

Vecino y MiembroDirectiva extienden Usuario: comparten la identidad y las credenciales, y cada una agrega su atributo propio (número de casa o cargo, respectivamente). De igual forma, las plantillas de correo extienden PlantillaEmail: bienvenida, solicitud recibida, solicitud respondida, comunicado, cuota pendiente, pago registrado y asignación de proyecto. La estructura HTML común se define en la clase base, y cada subclase solo especifica su asunto y su contenido.

Polimorfismo

Usuario declara el método `getRol()` sin implementarlo. Vecino devuelve `VECINO`, mientras que MiembroDirectiva devuelve el rol correspondiente a su cargo: presidente, secretario o tesorero. De este modo, tanto el filtro de seguridad como el frontend consultan el rol sin necesidad de conocer la subclase concreta. El correo funciona de la misma manera: EmailService recibe una PlantillaEmail y llama a `generar()`, y es la subclase concreta la que construye el asunto y el cuerpo. Así, el servicio no necesita una condición por cada tipo de mensaje.

Clases abstractas

Usuario es abstracta porque en el sistema no existe un usuario sin función: siempre es vecino o miembro de la directiva, y exigir la implementación de `getRol()` evita cuentas sin rol. PlantillaEmail también es abstracta, porque todo correo del sistema surge de un hecho concreto; por ello, `construirAsunto()` y `construirCuerpo()` deben implementarse en cada subclase. El método `generar()` se declara final para que todas las plantillas armen el mensaje de la misma forma.

Programación concurrente

Los hilos se aplican en el envío de correos, un caso real del sistema y no una tarea de demostración. Una conexión SMTP puede tardar varios segundos, más aún si hay reintentos; si ese trabajo se ejecutara en el hilo de la petición HTTP, el vecino tendría que esperar en la pantalla solo para que se envíe un aviso. Por eso, EmailServiceImpl utiliza un pool fijo de hilos (`Executors.newFixedThreadPool()`): la operación de negocio guarda el dato, encola el correo y responde de inmediato, mientras el envío continúa en segundo plano.

Cada envío se reintenta hasta tres veces, con una espera que aumenta entre intentos. El resultado se registra en RegistroEmail: estado, número de intentos, hilo que lo procesó y, en caso de fallo, el último error. El historial de correos utiliza un `ConcurrentHashMap`, ya que varios hilos actualizan registros al mismo tiempo; una colección común no sería segura en ese escenario, porque dos envíos simultáneos podrían sobrescribirse.

Los correos que se envían mediante este flujo, una vez que el dato se ha guardado correctamente, son: solicitud recibida, solicitud respondida, comunicado, cuota pendiente y pago registrado.

Colecciones

- `ArrayList` para los listados que se leen y escriben en cada archivo `.dat` (usuarios, proyectos, votos, cuotas, movimientos, actas, solicitudes, comunicados y comentarios), ya que conserva el orden de inserción y el volumen de datos de la asociación es acotado.
- List de AcuerdoActa dentro de Acta, porque los acuerdos se manejan junto con el acta y no se consultan por separado.
- `ConcurrentHashMap` para la bitácora de correos, indexado por id, de modo que el pool de hilos pueda actualizar los envíos sin bloquear todo el mapa.
- Set con los cargos autorizados en secretaría y en tesorería, para verificar en una sola consulta si un cargo tiene permiso de escritura.

Arquitectura por capas

El backend separa responsabilidades en capas. El controller recibe la petición HTTP y no contiene reglas de negocio. El service toma las decisiones: quién puede escribir en secretaría, si un voto está repetido o cómo se calcula el balance. El DAO se encarga del acceso al archivo .dat, y su interfaz (por ejemplo, UsuarioDAO o ProyectoDAO) evita que el service dependa de los detalles de serialización. Por último, la entidad representa lo que se almacena, mientras que el DTO define lo que entra y sale por la API, de modo que no se expone la contraseña ni la estructura interna de los archivos.

El frontend también está desacoplado del backend: Next.js no accede a los archivos .dat, sino que consume la API. Las Server Actions leen la cookie de sesión en el servidor y reenvían el token a Spring Boot, por lo que el JavaScript del navegador nunca tiene acceso al JWT.

Persistencia en archivos .dat

Cada agregado principal tiene su propio archivo: usuarios.dat, proyectos.dat, votos.dat, comentarios.dat, solicitudes.dat, comunicados.dat, actas.dat, cuotas.dat, movimientos.dat y emails.dat. Las clases que se almacenan implementan Serializable. El DAO lee la lista completa, aplica el cambio en memoria y vuelve a escribir el archivo. Para el volumen de datos de una sola asociación, esta estrategia resulta suficiente y cumple con la consigna del avance.

Versionamiento con Git

El frontend y el backend se encuentran en un mismo repositorio (monorepo), en las carpetas web-asociacion-comunal y backend-asociacion-comunal. Las funcionalidades se desarrollaron en ramas independientes (entre ellas feature/crud-usuario, feature/proyectos-crud, feature/jwt-comunidad-api y feature/envio-emails) y se integraron a main mediante Pull Requests que incluyen la descripción del cambio y un plan de pruebas. Los mensajes de commit siguen una convención: feat para nuevas funcionalidades, fix para correcciones y docs para documentación.

En este avance, el historial refleja el trabajo de los cuatro integrantes activos: Luis Gerardo Ramos Martínez aparece como Ezzz-Lui y como Luis Ramos; Marcos Armando Vásquez González, como Marcos6192005; Anderson Steven Flores Pérez, como Anderson-Flores-P; y Roberto De Jesus Erazo Ramirez, como Erazo93. Este documento se versiona en la raíz del repositorio, junto al del avance 1.

Conclusiones

En este avance, el equipo pasó del diseño planteado en el avance 1 a un sistema funcional. El vecino puede ingresar a su panel, votar y enviar solicitudes; la directiva administra proyectos, secretaría y tesorería; y los avisos por correo se envían en segundo plano.

La programación orientada a objetos se aplicó a las necesidades reales del problema y no como ejemplos aislados. La herencia permite modelar al vecino y a la directiva; el polimorfismo resuelve tanto el rol como el contenido de cada correo; los hilos responden a la necesidad de que el servidor SMTP no retrase al usuario; y las colecciones se eligieron según el tipo de acceso: listas para los archivos y un mapa concurrente para la bitácora de envíos.

Aunque el grupo sigue conformado formalmente por diez personas, en esta entrega trabajaron cuatro, y así se indica en el documento. El trabajo se repartió por módulos: arquitectura, autenticación, persistencia, tesorería, secretaría y votaciones; envío de correos con hilos; proyectos; y usuarios. Mantener los roles Scrum del avance 1 evita reestructurar la organización del grupo y, al mismo tiempo, deja constancia de quiénes desarrollaron este avance.

Queda pendiente la fecha del avance 3. Asimismo, la base de datos relacional queda fuera de esta entrega por consigna, ya que los archivos .dat cubren la persistencia requerida por ahora. Si el siguiente avance exige PostgreSQL u otro motor, el cambio se concentraría en la capa DAO, sin necesidad de reescribir las reglas de negocio implementadas en los services.

Bibliografía

- Schwaber, K. y Sutherland, J. (2020). La Guía de Scrum. Scrum.org. <https://scrumguides.org/>
- Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley.
- Oracle. (2024). The Java Tutorials: Concurrency y Object-Oriented Programming. <https://docs.oracle.com/javase/tutorial/>
- Broadcom. (2024). Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/documentation.html>
- Vercel. (2024). Next.js Documentation: App Router y Server Actions. <https://nextjs.org/docs>
- Guevara Gómez, D. E. (2026). Avance 2 proyecto. Programación II. Universidad Evangélica de El Salvador. Guía de entrega de este corte.